Name: Peggy Liow En Ki

Student ID: 23020852

Introduction

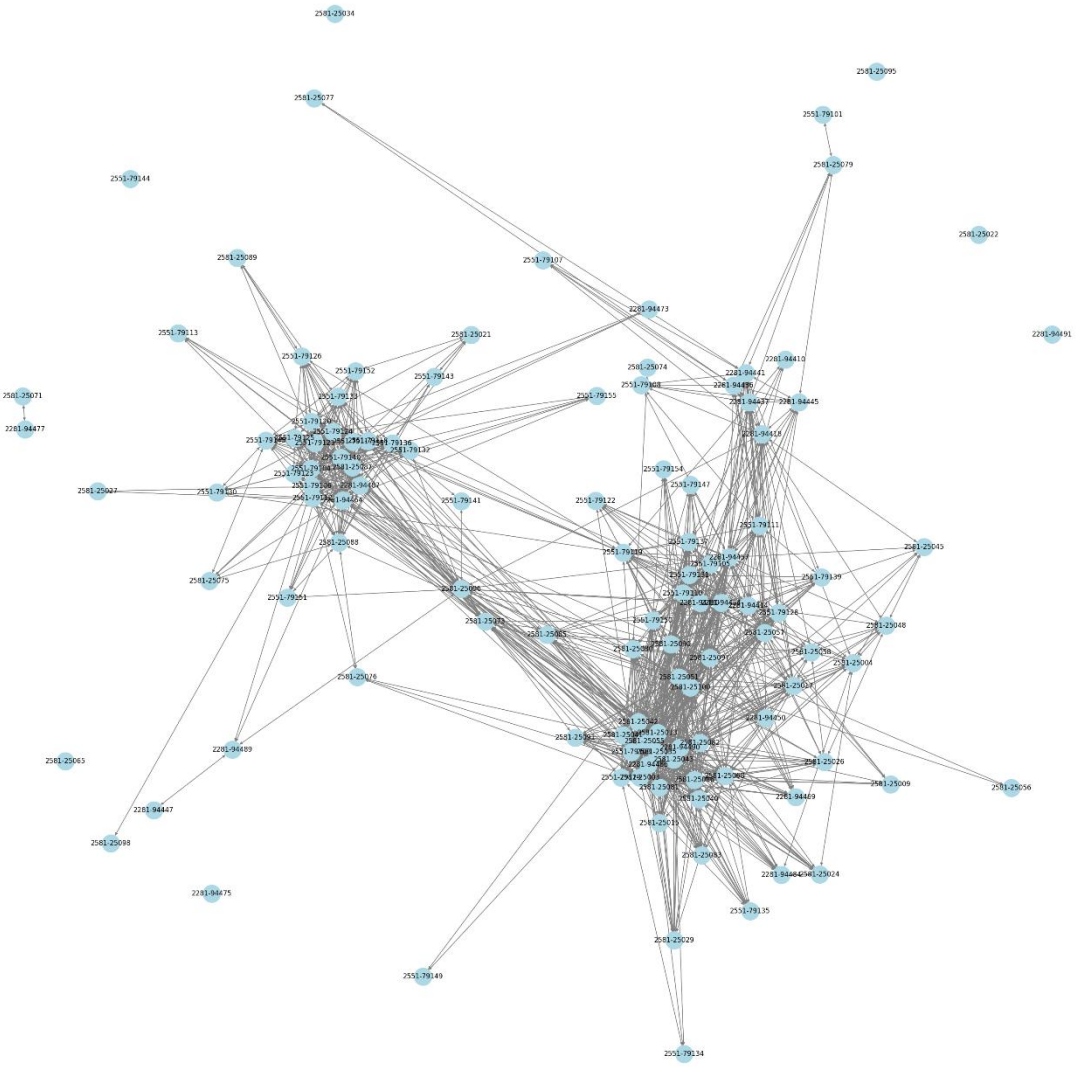
In this individual assignment, the dataset used pertains to the social interactions of house finches (Haemorhous mexicanus), which is collected by Pratha, Méndez and Bansal at Virginia Tech, Virginia, USA. They utilize the RFID technology to record feeding events and interactions at feeders, which can be analyzed to understand the social behaviour and the spread of contagious pathogens of house finches, such as Mycoplasma gallisepticum (MG). The network was generated based on patterns of co-occurrence by individuals in the same feeding events. Associations between birds were defined using the simple ratio index.

According to El-Gazzar (2020), he mentioned that Mycoplasma gallisepticum is commonly involved in polymicrobial “chronic respiratory disease” in Avian species. The morbidity rate is typically high and the mortality rate is low in affected flocks. It is usually transmitted via infectious aerosols, respiratory route and through contamination of feed, water and the environment. Once an individual bird or flocks are infected, they remain infected for life and act as carriers or reservoir for infection. Flock-to-flock transmission occurs through direct or indirect contact from the movement of birds, people or fomites from infected to susceptible flocks.

The dataset contains 117 nodes and 1027 edges. Each of the nodes represents a particular house finch and each edges represent the connections between these house finches. The key attribute of the dataset is node id. Below shows the snapshot of the dataset.

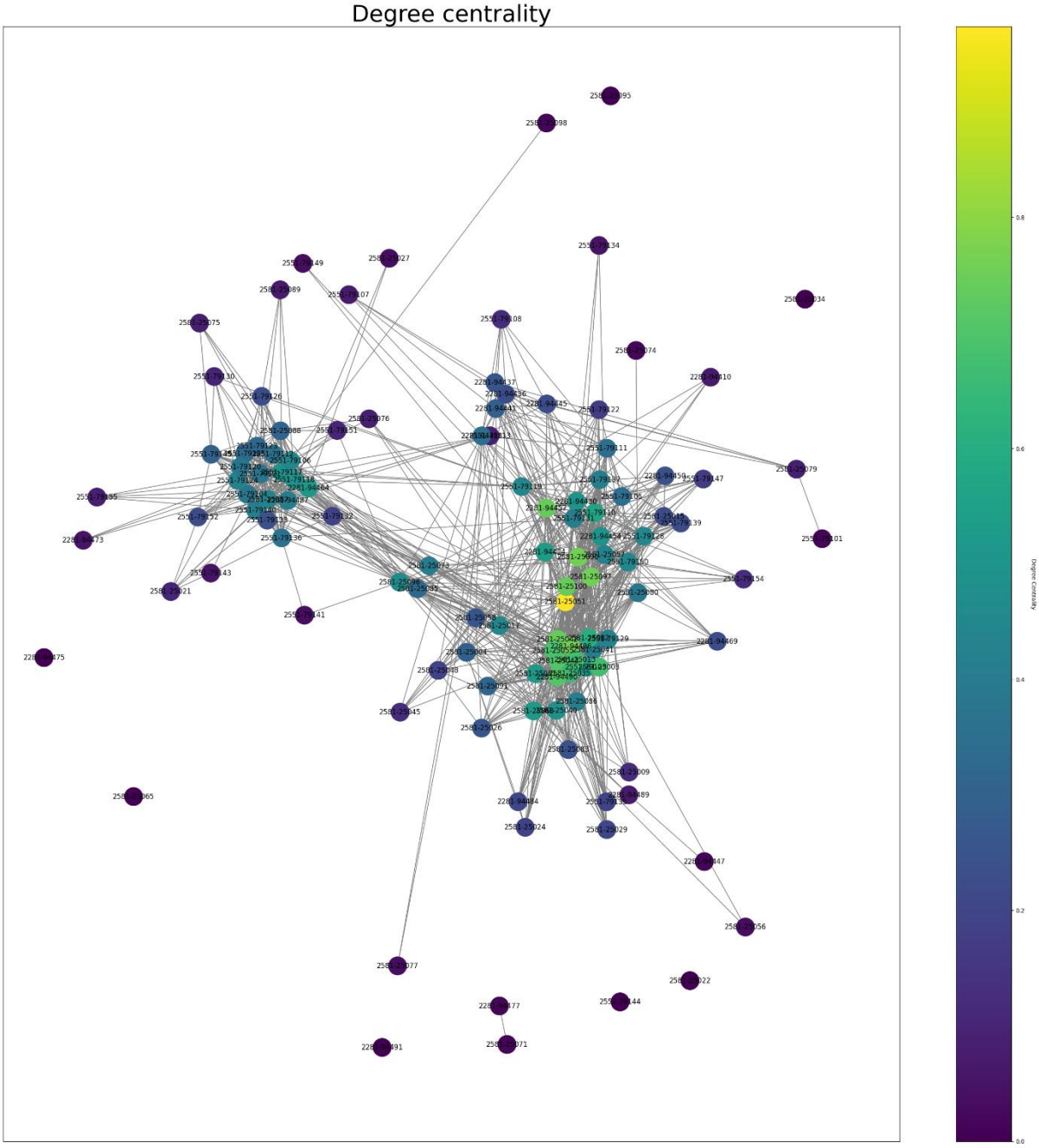
Node ID		
2581-25042		
2581-25043		
2581-25040		
2581-25041	2581-25082	
2581-25045	2581-25083	
2281-94450	2581-25080	
2281-94457	2581-25081	
2281-94454	2581-25088	
2551-79141	2581-25089	
2551-79140	2551-79108	
2551-79143	2551-79105	
2581-25034	2551-79104	
2551-79145	2551-79107	
2551-79144	2551-79106	
2551-79147	2551-79101	2551-79120
2551-79149	2581-25035	2281-94484
2581-25048	2551-79103	2281-94487
2581-25051	2581-25079	2281-94486
2581-25055	2581-25073	2551-79150
2581-25057	2581-25071	2551-79129
2581-25056	2581-25077	2551-79128
2281-94441	2581-25076	2581-25015
2581-25058	2581-25075	2581-25017
2281-94445	2581-25074	2581-25013
2281-94447	2581-25095	2551-79154
2581-25065	2581-25097	2551-79155
2281-94475	2581-25096	2551-79123
2281-94477	2581-25091	2551-79152
2581-25068	2581-25090	2581-25021
2281-94473	2581-25098	2581-25022
2281-94464	2551-79130	2551-79151
2281-94469	2551-79131	2581-25024
2551-79118	2551-79132	2581-25026
2551-79119	2551-79133	2581-25027
2551-79117	2551-79134	2281-94430
2551-79112	2551-79135	2581-25029
2551-79113	2551-79136	2281-94436
2551-79110	2551-79137	2281-94437
2551-79111	2551-79139	
2581-25100	2281-94490	
2281-94414	2281-94491	
2281-94410	2581-25009	
2281-94418	2581-25004	
2581-25086	2581-25003	
2581-25087	2551-79126	
2581-25085	2551-79125	
	2551-79124	
	2281-94489	
	2551-79122	
	2551-79121	

Network Visualization



Upon initial analysis, the network appears to have two dense clusters, suggesting two main groups within the network. The nodes within each cluster are more densely connected to each other than to nodes outside the cluster. The central nodes in each cluster might be more important due to their position and number of connections. There are 7 nodes that is isolated and not connected to the main network, which indicates entities with no relationships to others in the network. This initial exploration shows that certain house finches within the network are more central and connected to others. Next, I will conduct a more detailed analysis of these key nodes and explore potential insights in the network.

Degree Centrality



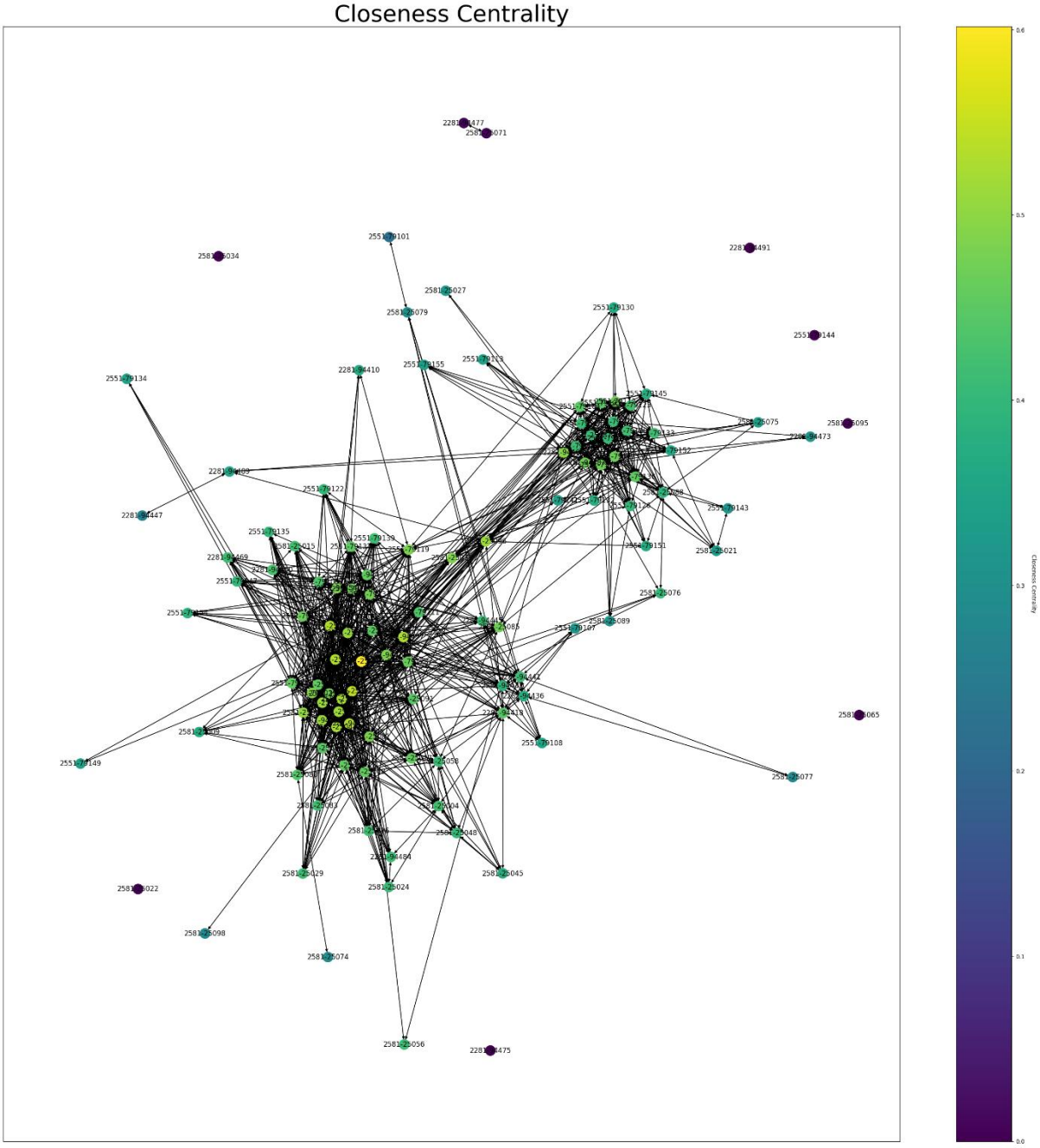
Node Number	Node ID	Degree Centrality
17	2581-25051	0.482759
96	2281.25051	0.379310
71	2581-25090	0.379310
68	2581-25097	0.379310
0	2581-25042	0.370690

According to Zhang and Luo (2017), degree centrality is measured by the total amount of direct links with other nodes. This indicates that the larger the number of connections to other nodes, the higher the importance of a node within a network. Based on the colour bar in the degree centrality diagram above, the brighter colours indicates higher degree centrality, whereas the darker colours indicate low degree centrality. Based on the table above, it shows

that the node “2581-25051” has the highest centrality as compared to the other nodes. This node could represent an individual bird which has numerous social interactions. This bird might be an influential member of its social group, which potentially affects the movement or behaviour patterns of other birds.

The results indicates that the node “2581-25051”, which has high centrality would be a potential factor of spreading Mycoplasma gallisepticum. This node interacts with many others, if it was infected, it could facilitate the rapid spread of this disease throughout the network. However, this node might be a key target for disease management interventions. Monitoring, isolating or treating nodes with high degree centrality could help in controlling the spread of disease within the network.

Closeness Centrality



Node Number	Node ID	Closeness Centrality
17	2581-25051	0.601819
0	2581-25042	0.542298
82	2281-94490	0.533504
71	2581-25090	0.533504
39	2581-25100	0.530636

Based on Zhang and Luo (2017), closeness centrality is measure of one node to the other nodes' sum distances. If the distance between one node and the other node is small, then the node has a high closeness centrality. It also means that the larger the value of closeness centrality, the more central and closer the nodes are displayed in the network. The average closeness centrality of this diagram is 0.3891, which indicates that the nodes in the

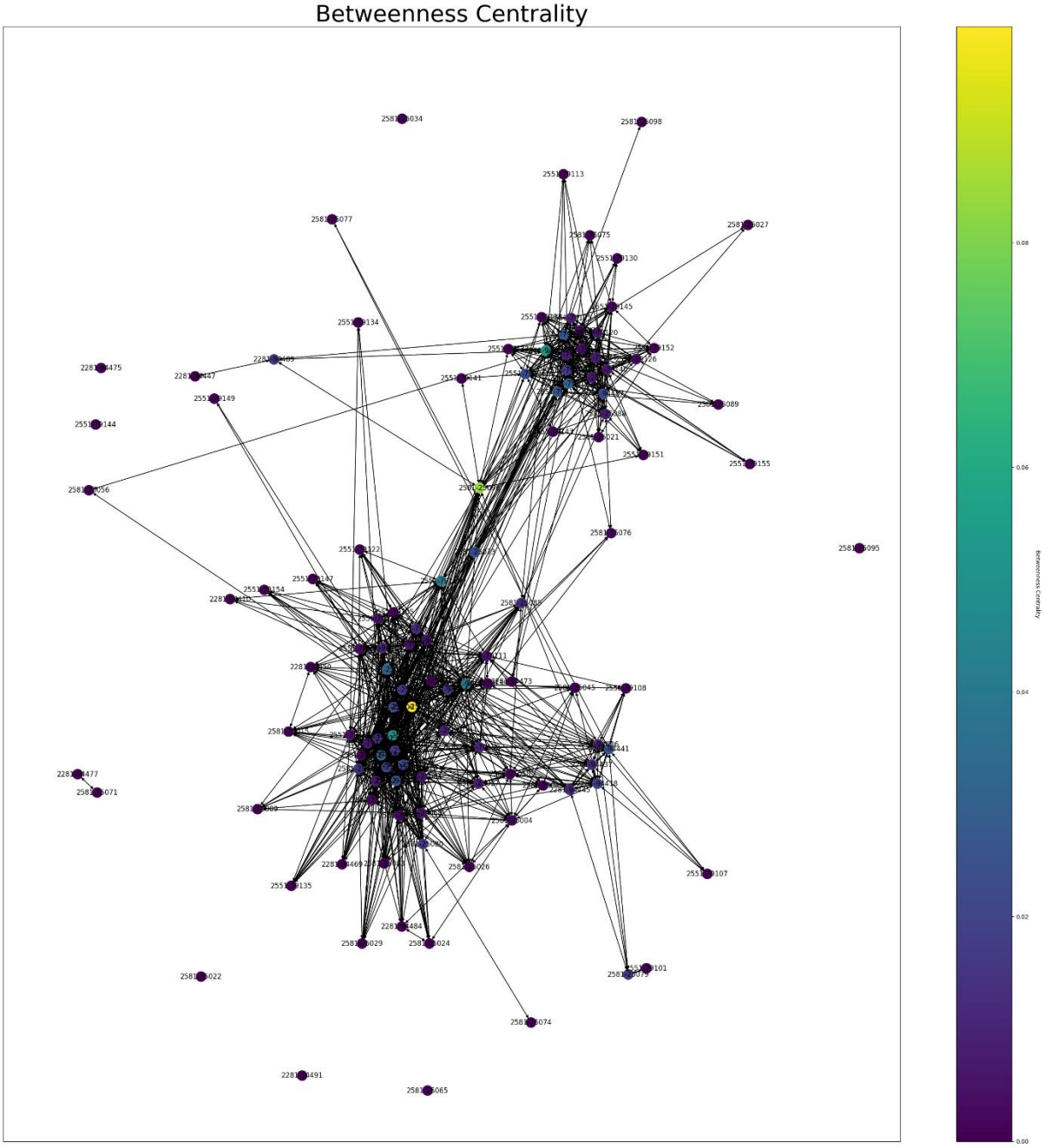
network are relatively close to each other, most nodes are not too far away from each other in terms of short path distances.

According to the table above, it shows that node “2581-25051” has the highest value of closeness centrality among the listed nodes. This indicates that node “2581-25051” can reach to other nodes quickly. Nodes with higher value of closeness centrality might be important for the spread of pathogens, which facilitate rapid transmission of disease through the network. However, there are 7 nodes that has a closeness centrality of 0. This indicates that they are poorly connected to the network.

The results indicate that the node “2581-25051” should be the focal point for monitoring and intervention efforts. Nodes with high closeness centrality are the key to quick disease transmission and they should be managed properly to control the spread of Mycoplasma gallisepticum**.** For nodes with a closeness centrality of 0 are poorly connected to the rest of the network. They might not significantly affect the spread of Mycoplasma gallisepticum, but they still have the risk of interacting with other network components.

Based on Behboudi (2022), in terms of prevention and control, biosecurity and sanitation are essential to prevent the spread of Mycoplasma gallisepticum into a susceptible flock. Biosecurity should keep wild birds away from the flock and strict sanitation procedures should be applied. Vaccines can also help to control the transmission of Mycoplasma gallisepticum within the flocks.

Betweenness Centrality



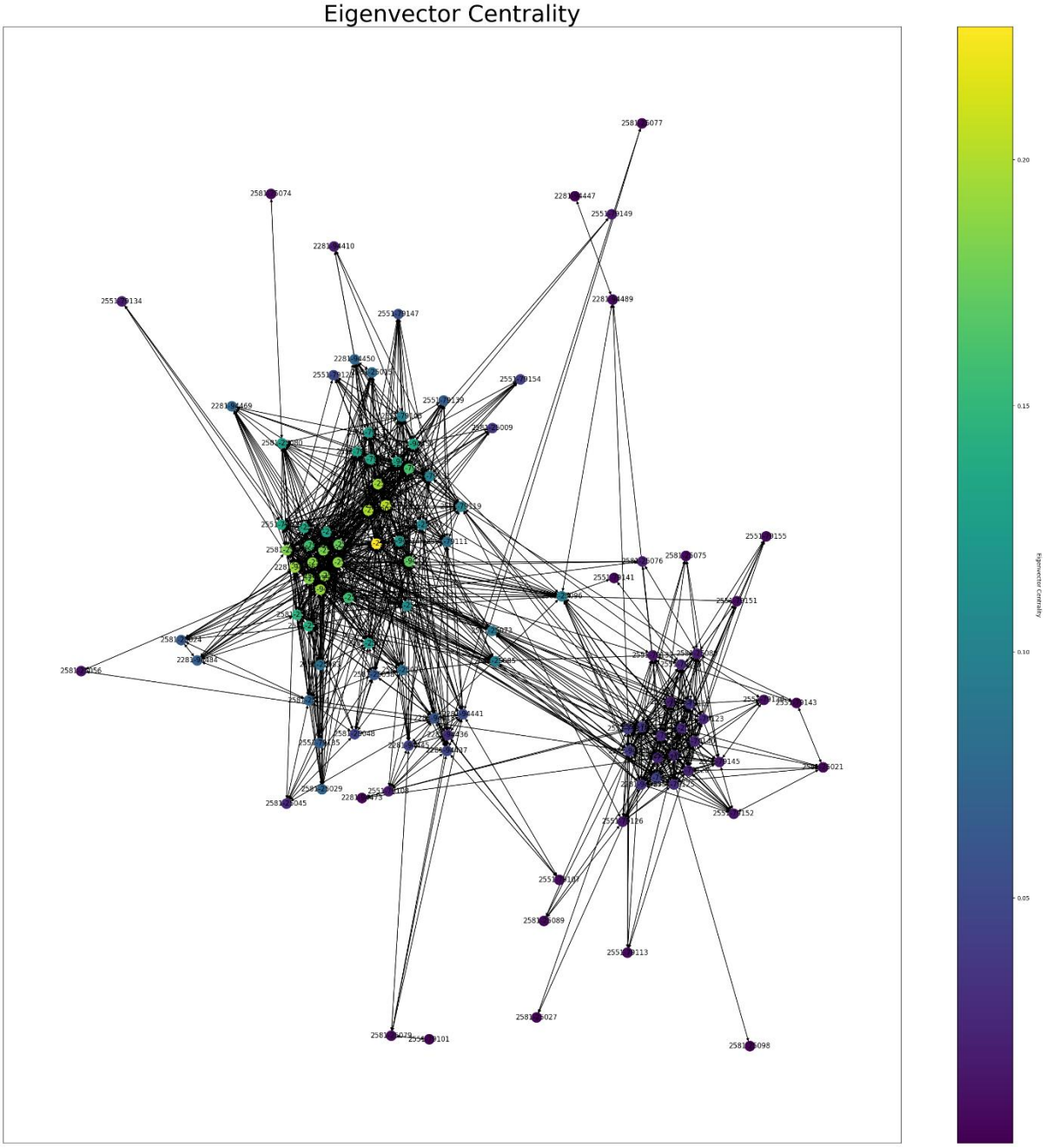
Node Number	Node ID	Betweenness Centrality
17	2581-25051	0.099219
69	2581-25096	0.084298
30	2281-94464	0.051911
0	2581-25042	0.050053
33	2551-79119	0.040286
…		
108	2581-25022	0.000000
114	2581-25029	0.000000
112	2581-25027	0.000000

104	2551-79155	0.000000
15	2551-79149	0.000000
24	2281-94447	0.000000
83	2281-94491	0.000000
13	2551-79144	0.000000
36	2551-79133	0.000000
41	2281-94410	0.000000
29	2281-94473	0.000000
51	2581-25089	0.000000
27	2281-94477	0.000000
55	2551-79107	0.000000
11	2581-25034	0.000000
57	2551-79101	0.000000
26	2281-94475	0.000000
62	2581-25071	0.000000
63	2581-25077	0.000000
65	2581-25075	0.000000
66	2581-25074	0.000000
67	2581-25095	0.000000
20	2581-25056	0.000000
72	2581-25098	0.000000
14	2551-79147	0.000000
77	2551-79134	0.000000
25	2581-25065	0.000000

Zhang and Luo (2017) mentioned that betweenness centrality is the measure of one node which serves as a ‘mediation’ role in the network. If one node locates in the only way which others nodes have to go through like connection, then this node should be important and very likely have high betweenness centrality. Based on the table above, it shows that the node “2581-25051” has the highest betweenness centrality among other nodes, which indicates that it might serves as a significant role across the network. However, there are 27 nodes with the value 0 in betweenness centrality.

The results indicate that node “2581-25051” lies on the shortest paths between many pairs of other nodes, making it a critical connector. If this node is infected, it could facilitate the disease’s movement across different parts of the network. However, the 27 nodes with betweenness centrality of 0 are not part of the shortest paths connecting other nodes, they do not have a central role in network connectivity. Although, these nodes are less central, they should be still monitored for any potential interactions or disruptions. A comprehensive monitoring strategy should account for all nodes.

Eigenvector Centrality

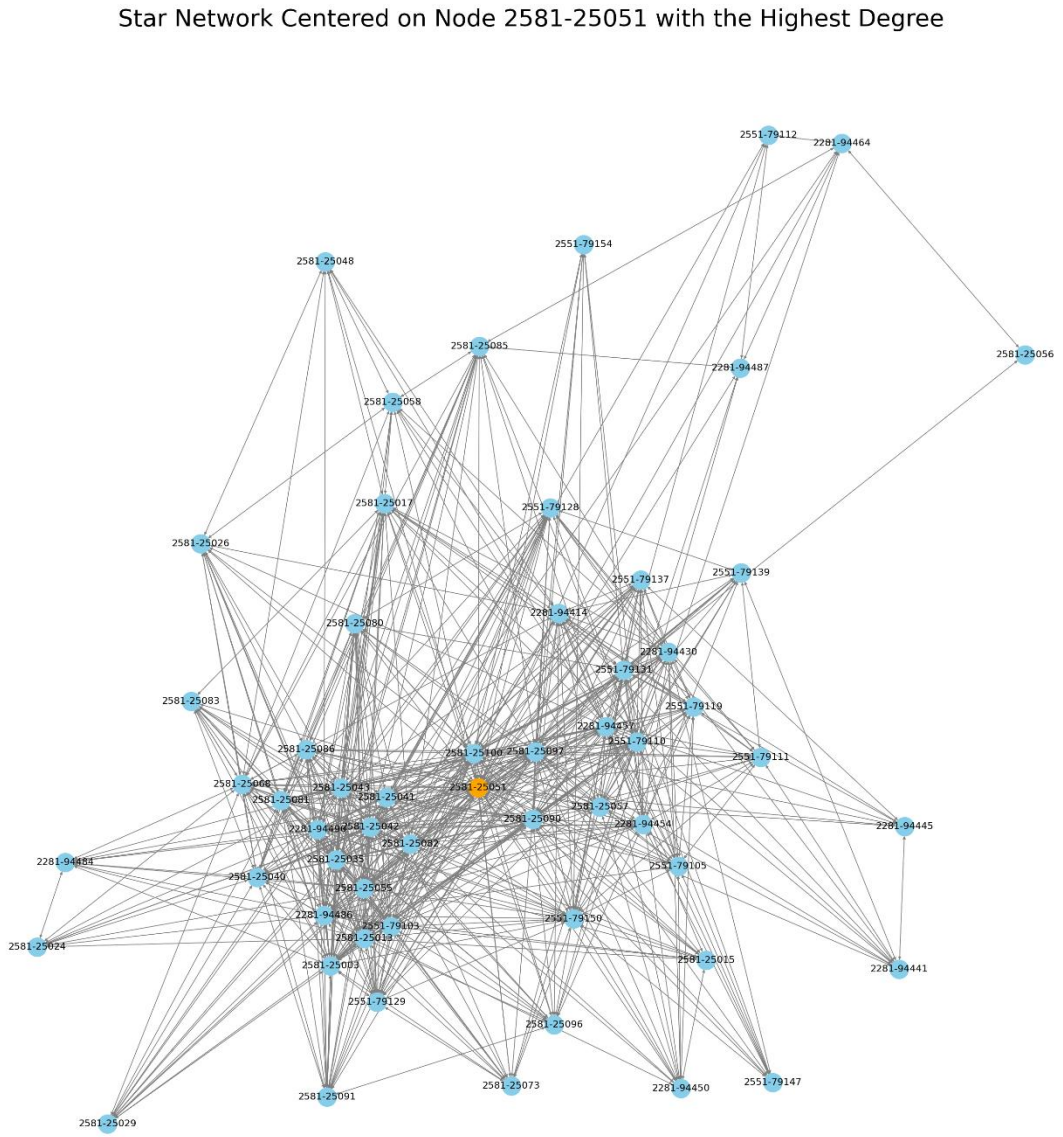


Node Number	Node ID	Eigenvector Centrality
15	2581-25051	0.227043
61	2581-25097	0.203064
34	2581-25100	0.198884
64	2581-25090	0.195480
88	2281-94486	0.192600

Based on Bihari and Pandia (2015), eigenvector centrality is the measure of the influence of a node in a network. Eigenvector centrality not only consider the total number of adjacent nodes, but also consider the importance of adjacent node. In eigenvector centrality, all connections are not equal. It is calculated by accessing how well connected an individual to the parts of the network with the greatest connectivity.

The above eigenvector centrality diagram shows that the node “2581-25051” has the highest value of eigenvector centrality, which is 0.227043. The results indicates that nodes with higher value of eigenvector centrality have substantial influence due to its connections with other important nodes. These nodes are likely to be the major player in the transmission of diseases. Its connections to other well-connected nodes make it a central point for the transmission of pathogens.

Star Network




```
Adjacency Matrix:
[[0.         0.00269542 0.0099728  ... 0.02008032 0.         0.00948767]
 [0.00269542 0.         0.0023753  ... 0.         0.         0.00269542]
 [0.0099728  0.0023753  0.         ... 0.02058824 0.         0.03342618]
 ...
 [0.02008032 0.         0.02058824 ... 0.         0.         0.05228758]
 [0.         0.         0.         ... 0.         0.         0.        ]
 [0.00948767 0.00269542 0.03342618 ... 0.05228758 0.         0.        ]]
Degrees of each node: {'2581-25097': 86, '2281-94450': 28, '2581-25090': 78, '2581-25024': 20, '2551-79147': 20, '2581-25040': 54, '2581-25057': 42, '2581-25043': 72, '2281-94445': 
Total number of links: 1276
Central Node: 2581-25051, Degree: 112
```

The above diagram shows the star network centered on node “2581-25051”, which has the highest degree of 112. The above diagram has a 1276 total number of links. This node is directly connected to all other nodes, it has the highest degree centrality and closeness centrality as it can quickly reach all other nodes in just one step. This node is vital for the network’s connectivity and efficiency, facilitating rapid spread of resources or pathogens.

The results emphasize the importance of node “2581-25051” due to its central role. It is vital for maintaining the network’s overall connectivity. This extensive number of direct links ensures that it is integral to the network’s structure and function. It will facilitate the rapid spread of Mycoplasma gallisepticum throughout the network. This star diagram also indicates

that this node should be the primary focus for monitoring and intervention, which is important in ensuring the health and stability of this node to control the spread of disease.

According to Mugunthan et al. (2023), Mycoplasma gallisepticum has two routes of transmission, which are vertical transmission or horizontal transmission. Vertical transmission may happen in egg and in embryo and transmission to the progeny possibly happens due to the consequence of respiratory infection of hens. The vaccine strain ts-11 is capable of vertical transmission. For horizontal transmission, it may occur by respiratory aerosols, direct contact with infected birds or indirect modes. An example of the factor in indirect transmission is infection through contaminated food. Regular cleaning and disinfection could reduce the spread of this organism.

Code

```
# -*- coding: utf-8 -*-
"""code_23020852

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/1KvqLCG7ZsU2szYdgMNZi2pYrGYXdUNgH
"""

!pip install networkx

import networkx as nx
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

#Import Files
from google.colab import files
uploaded = files.upload()

# Read the graph from the file
G=
nx.read_graphml('Aves_songbird_association_weighted_weighted_adelman_social_so
ngbird.graphml')
# dataset https://doi.org/10.7910/DVN/N5YHLL/YZRNWC

print(G)

plt.figure(figsize=(30,30))  # Set the figure size

pos = nx.spring_layout(G)  # Positioning the nodes using spring layout

nx.draw(
    G,
    pos,
    with_labels=True,
    node_size = 1000,
    node_color = 'lightblue',
    font_size = 12,
    font_color = 'black',
    edge_color = 'gray'
    )

plt.title("Network Visualization", fontsize = 40)
plt.show()

nodes_data = []
```

```python
# Iterate through the nodes and collect their data
for node, data in G.nodes(data=True):
    node_info = {'Node ID': node}
    node_info.update(data)
    nodes_data.append(node_info)

# Create a DataFrame from the list of dictionaries
df = pd.DataFrame(nodes_data)

# Set display options to show the full DataFrame
pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
pd.set_option('display.max_colwidth', None)

# Display the DataFrame
print(df)

# Export the DataFrame to a CSV file
df.to_csv('Aves_songbird_association_weighted_weighted_adelman_social_songbird
.csv', index=False)

# Degree centrality
degree_centrality = nx.degree_centrality(G)

# Create a color map based on degree centrality values
norm = plt.Normalize(vmin=min(degree_centrality.values()),
vmax=max(degree_centrality.values()))
node_colors = plt.cm.viridis(norm(list(degree_centrality.values())))

#Visualize degree centrality
plt.figure(figsize=(35, 35))
pos = nx.spring_layout(G)

nodes = nx.draw_networkx_nodes(
    G, pos,
    node_size=1000,
    node_color = list(degree_centrality.values()),
)
edges = nx.draw_networkx_edges(G, pos, edge_color='gray')
labels = nx.draw_networkx_labels(G, pos, font_size=12, font_color='black')

# Add a color bar
sm = plt.cm.ScalarMappable(cmap=plt.cm.viridis, norm=norm)
sm.set_array([])
cbar = plt.colorbar(sm)
cbar.set_label('Degree Centrality', rotation=270, labelpad=20)
```

```
plt.title("Degree centrality", fontsize = 40)
plt.show()

"""
Degree centrality is a measure of the number of connections a node has in a
network. It indicates how well-connected or popular a node is within the
network.
"""

# Compute degree centrality
degree_centrality = nx.degree_centrality(G)

# Create a DataFrame to tabulate the degree centrality values
df = pd.DataFrame(list(degree_centrality.items()), columns=['Node ID', 'Degree
Centrality'])

# Sort the DataFrame by 'Degree Centrality' in descending order
df_sorted = df.sort_values(by='Degree Centrality', ascending=False)

# Set Pandas display option to show all rows
pd.set_option('display.max_rows', len(df_sorted))

# Display the sorted DataFrame
print(df_sorted)

#Closeness centrality
closeness_centrality = nx.closeness_centrality(G)

#Visualize closeness centrality
plt.figure(figsize=(35, 35))
nx.draw_networkx(G, node_color= list(closeness_centrality.values()))

# Create a color map based on closeness centrality values
norm = plt.Normalize(vmin=min(closeness_centrality.values()),
vmax=max(closeness_centrality.values()))
node_colors = plt.cm.viridis(norm(list(closeness_centrality.values())))

# Add a color bar
sm = plt.cm.ScalarMappable(cmap=plt.cm.viridis, norm=norm)
sm.set_array([])
cbar = plt.colorbar(sm)
cbar.set_label('Closeness Centrality', rotation=270, labelpad=20)

plt.title("Closeness Centrality", fontsize = 40)
plt.show()

"""
```

```
Closeness centrality measures how close a node is to other nodes in the
network in terms of the shortest path length
"""

# Compute closeness centrality
closeness_centrality = nx.closeness_centrality(G)

# Create a DataFrame to tabulate the closeness centrality values
df_closeness = pd.DataFrame(list(closeness_centrality.items()), columns=['Node
ID', 'Closeness Centrality'])

average_closeness = df_closeness['Closeness Centrality'].mean()
print("Average Closeness Centrality:", average_closeness)

# Sort the DataFrame by 'Closeness Centrality' in descending order
df_closeness_sorted = df_closeness.sort_values(by='Closeness Centrality',
ascending=False)

# Set Pandas display option to show all rows
pd.set_option('display.max_rows', len(df_sorted))

# Display the sorted DataFrame
print(df_closeness_sorted)

#Betweeness Centrality
betweenness_centrality = nx.betweenness_centrality(G)

#Visualize betweeness centrality
plt.figure(figsize=(35, 35))
nx.draw_networkx(G, node_color= list(betweenness_centrality.values()))

# Create a color map based on closeness centrality values
norm = plt.Normalize(vmin=min(betweenness_centrality.values()),
vmax=max(betweenness_centrality.values()))
node_colors = plt.cm.viridis(norm(list(betweenness_centrality.values())))

# Add a color bar
sm = plt.cm.ScalarMappable(cmap=plt.cm.viridis, norm=norm)
sm.set_array([])
cbar = plt.colorbar(sm)
cbar.set_label('Betweenness Centrality', rotation=270, labelpad=20)

plt.title("Betweenness Centrality", fontsize = 40)
plt.show()

"""
```

```
Betweenness centrality measures how often a node lies on the shortest paths
between other nodes in the network. It quantifies the influence a node has in
connecting different parts of the network.

"""

# Compute Betweenness centrality
betweenness_centrality = nx.betweenness_centrality(G)

# Create a DataFrame to tabulate the betweenness centrality values
df_betweenness = pd.DataFrame(list(betweenness_centrality.items()),
columns=['Node ID', 'Betweenness centrality'])

# Sort the DataFrame by 'Betweenness centrality' in descending order
df_betweenness_sorted = df_betweenness.sort_values(by='Betweenness
centrality', ascending=False)

# Set Pandas display option to show all rows
pd.set_option('display.max_rows', len(df_betweenness_sorted))

# Display the sorted DataFrame
print(df_betweenness_sorted)

# Save the DataFrame to a CSV file, ensuring the 'Node ID' is included
df_betweenness_sorted.to_csv('betweenness_centrality.csv', index=True)

#Eigenvector centrality
#Try to load the graph from GraphML file
G_multi =
nx.read_graphml('Aves_songbird_association_weighted_weighted_adelman_social_so
ngbird.graphml')
# Convert to a directed graph if it is a multigraph
if G_multi.is_multigraph():
    G = nx.DiGraph()  # Initialize a directed graph
    for u, v, key, data in G_multi.edges(keys=True, data=True):
        G.add_edge(u, v, **data)
else:
    G = nx.DiGraph(G_multi)  # Convert to directed graph

# Ensure G is a directed graph
if not G.is_directed():
    G = G.to_directed()

# Get the largest strongly connected component
strongly_connected_components = list(nx.strongly_connected_components(G))
largest_scc = max(strongly_connected_components, key=len)
G_largest = G.subgraph(largest_scc).copy()
```

```python
# Calculate eigenvector centrality with more iterations
eigenvector_centrality = nx.eigenvector_centrality(G_largest, max_iter=200)

# Visualize eigenvector centrality
plt.figure(figsize=(35, 35))
nx.draw_networkx(G_largest, node_color=list(eigenvector_centrality.values()))

# Create a color map based on eigenvector centrality values
norm = plt.Normalize(vmin=min(eigenvector_centrality.values()),
vmax=max(eigenvector_centrality.values()))
node_colors = plt.cm.viridis(norm(list(eigenvector_centrality.values())))

# Add a color bar
sm = plt.cm.ScalarMappable(cmap=plt.cm.viridis, norm=norm)
sm.set_array([])
cbar = plt.colorbar(sm)
cbar.set_label('Eigenvector Centrality', rotation=270, labelpad=20)

plt.title("Eigenvector Centrality", fontsize=40)
plt.show()

"""
Eigenvector centrality measures the importance of a node based on both its
direct connections and the centrality of its neighbours. It considers the
quality of a node's connections rather than just the quantity
"""
# Convert to a directed graph if it is a multigraph
if G_multi.is_multigraph():
    G = nx.DiGraph()  # Initialize a directed graph
    for u, v, key, data in G_multi.edges(keys=True, data=True):
        G.add_edge(u, v, **data)
else:
    G = nx.DiGraph(G_multi)  # Convert to directed graph

# Ensure G is a directed graph
if not G.is_directed():
    G = G.to_directed()

# Get the largest strongly connected component
strongly_connected_components = list(nx.strongly_connected_components(G))
largest_scc = max(strongly_connected_components, key=len)
G_largest = G.subgraph(largest_scc).copy()


# Calculate eigenvector centrality with more iterations
eigenvector_centrality = nx.eigenvector_centrality(G_largest, max_iter=200)

# Create a DataFrame to tabulate the eigenvector centrality values
```

```
df_eigenvector = pd.DataFrame(list(eigenvector_centrality.items()),
columns=['Node ID', 'Eigenvector Centrality'])

# Sort the DataFrame by 'Eigenvector Centrality' in descending order
df_eigenvector_sorted = df_eigenvector.sort_values(by='Eigenvector
Centrality', ascending=False)

# Display the sorted DataFrame
print(df_eigenvector_sorted)

# Full Dataframe

# Calculate centrality measures on the largest component
degree_centrality = nx.degree_centrality(G)
closeness_centrality = nx.closeness_centrality(G)
betweenness_centrality = nx.betweenness_centrality(G)
eigenvector_centrality = nx.eigenvector_centrality(G, max_iter=200)

# Prepare a list of dictionaries to hold the nodes data including centrality
measures
nodes_data = []

# Iterate through the nodes and collect their data
for node, data in G.nodes(data=True):
    node_info = {'Node ID': node}
    node_info.update(data)
    node_info['Degree Centrality'] = degree_centrality.get(node, 0)
    node_info['Closeness Centrality'] = closeness_centrality.get(node, 0)
    node_info['Betweenness Centrality'] = betweenness_centrality.get(node, 0)
    node_info['Eigenvector Centrality'] = eigenvector_centrality.get(node, 0)
    nodes_data.append(node_info)

# Create a DataFrame from the list of dictionaries
full_df = pd.DataFrame(nodes_data)

# Format the centrality measures to 4 decimal places
full_df['Degree Centrality'] = full_df['Degree
Centrality'].map('{:.6f}'.format)
full_df['Closeness Centrality'] = full_df['Closeness
Centrality'].map('{:.6f}'.format)
full_df['Betweenness Centrality'] = full_df['Betweenness
Centrality'].map('{:.6f}'.format)
full_df['Eigenvector Centrality'] = full_df['Eigenvector
Centrality'].map('{:.6f}'.format)

# Set display options to show the full DataFrame
pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)
```

```python
pd.set_option('display.width', None)
pd.set_option('display.max_colwidth', None)

# Display the DataFrame
print(full_df)

# Export the DataFrame to a CSV file
full_df.to_csv('graph_centrality_measures.csv', index=True)

#star network
# Identify the node with the highest degree (number of connections)
degrees = dict(G.degree())
central_node = max(degrees, key=degrees.get)

# Extract the star network centered on the central node
star_network = nx.ego_graph(G, central_node, radius=1)

# Draw the star network
plt.figure(figsize=(28, 28))
pos = nx.spring_layout(star_network)
nx.draw(star_network, pos, with_labels=True, node_color="skyblue",
edge_color="gray", node_size=1000, font_size=16)
nx.draw_networkx_nodes(star_network, pos, nodelist=[central_node],
node_color="orange", node_size=1000)

plt.title(f"Star Network Centered on Node {central_node} with the Highest
Degree", fontsize=40)
plt.show()

# Show the Adjacency Matrix
print("Adjacency Matrix:")
print(nx.adjacency_matrix(star_network).todense())

# Show the Degree of each node
print("Degrees of each node:", dict(star_network.degree()))

# Show the total number of links
print("Total number of links:", star_network.number_of_edges())

# Show the central node
print(f"Central Node: {central_node}, Degree: {degrees[central_node]}")
```

Individual reflection

This individual reflection is regarding the group and individual assignments assigned by our lecturer last month. We were required to perform three types of analysis on the datasets we chose. Two for the group assignment and one for the individual assignment. We were in a team of 4 and we carried out location analysis and text analysis. For my individual assignment, I performed network analysis on a dataset chosen related to the social behaviour and spread of contagious pathogens of house finches, such as Mycoplasma gallisepticum (MG). The completion of both assignments was successful because everyone performed their tasks responsibly.

Throughout the individual assignment, I encountered unexpected difficulties. One of the difficulties was finding a suitable dataset. I had to explore the datasets by downloading and visualizing them to determine their suitability for network analysis. The file formats included GraphML, XLSX, CSV and Data, which required me to search online for methods to extract data from these files and import them into Google Colaboratory. Besides, some of the datasets were too large or too small, making it difficult to conduct the analysis work because visualizing the graph was challenging with an excessively large dataset. Moreover, I struggled to identify what the nodes represented. In the group assignment, the chosen dataset only provided information about node id and their x and y coordinates. We had to input these coordinates into the Nominatim website to identify the locations of the nodes, which represented the intersections of the streets in Downtown, Honolulu.

During the process of searching for datasets, I gained exposure to various tools and techniques. Exploring datasets on the Harvard Dataverse repository allowed me to search for high-quality datasets. I learned to use the filter function to search for datasets quickly. It is crucial for any analysis to have a reliable source for acquiring data. Additionally, I improved my skills in using Google Colaboratory for data analysis, coding, and visualizing data, as well as identifying problems and providing solutions to solve those problems. During the coding process, I often encountered syntax errors. I would check through my code for typos to solve the problem. If I couldn't resolve the issue, I would seek help and advice from my groupmates.

I referred to the tutorials while completing the coding network analysis section of the individual assignment. The main differences between the tutorial codes and mine is I have added a colour bar and colour map for degree centrality, closeness centrality, betweenness centrality and eigenvector centrality. I included code to adjust the font, title and node size colour. Besides, I wrote code to display the full DataFrame, including details like node ID and the values of degree centrality, eigenvector centrality, betweenness centrality and closeness centrality. For eigenvector centrality network, I added code to convert the graph to a directed graph if it is a multigraph, and to extract the largest strongly connected component. For the star network, I customized the figure size, node colour and the edge colour.

In the group assignments, all of us cooperated to finish our work on time. At first, our group leader, Isabelle, divided the work among us. She also set a timeframe by which we all had to finish our parts. Isabelle and I were in charge of the location analysis part, while the other two members handled the text analysis part. After completing our parts, we reviewed our documents and code before submission.

In my opinion, I have improved my problem-solving adaptability, which is an important criterion for becoming a better analyst. Whether it was dealing with syntax errors or finding suitable datasets, I learned to approach problems and seek ways to solve them. This adaptability is vital for tackling unexpected issues efficiently. Besides, time management is important for becoming a better analyst. Balancing multiple tasks and completing them within

the timeframe taught me the importance of planning and prioritizing my work. I was able to manage my time better by breaking down tasks and setting milestones. Moreover, analyzing datasets and interpreting results improved my critical thinking skills. I had to understand complex relationships within the data and draw meaningful conclusions from our analysis, which helped me think critically and make decisions based on data-driven insights. In the future, I hope to utilize and improve upon the things I have learned throughout the coursework, whether in assignments or in the workplace.

References

Behboudi, S. (2022). Avian mycoplasmosis (Mycoplasma gallisepticum). *CABI Compendium.* https://doi.org/10.1079/cabicompendium.92922

Bihari, A., & Pandia, M. K. (2015). Eigenvector Centrality and its Application in Research Professionals' Relationship Network. *1st International Conference on Futuristic Trends on Computational Analysis and Knowledge Management (ABLAZE)*, 510-514. IEEE.

El-Gazzar, M. (2020, May). *Mycoplasma gallisepticum Infection in Poultry.* MSD MANUAL Veterinary Manual. https://www.msdvetmanual.com/poultry/mycoplasmosis/mycoplasma-gallisepticum-infection-in-poultry

Mugunthan, S. P., Kannan, G., Chandra, H. M., & Paital, B. (2023). Infection, Transmission, Pathogenesis and Vaccine Development against Mycoplasma gallisepticum. *Vaccines.* https://doi.org/10.3390%2Fvaccines11020469

Zhang, J., & Luo, Y. (2017). Degree centrality, betweenness centrality, and closeness centrality in social network. *2nd International Conference on Modelling, Simulation and Applied Mathematics (MSAM2017),132,* 300-303.